Reviewer Pack — Minimal Tropical Hodge Index vs AC0 Parity Circuit Size

Entry f9751ccef221 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 15:50:54 UTC

Minimal Tropical Hodge Index vs AC0 Parity Circuit Size

Entry ID: `f9751ccef221` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 15:50:54 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Complexity Theory: AC0 Parity Circuit Complexity

Statement:

> ['For any AC0 parity circuit C with n inputs, the minimal tropical Hodge index of its associated tropical variety is at least Ω(log^2(n))□', 'The tropical Hodge index of the tropical variety associated with an AC0 circuit computing a function that does not compute parity on all possible inputs is at most O(log(n)).', 'The difference between the maximal and minimal tropical Hodge indices among all AC0 circuits for computing parity is Θ(log^2(n)).']

Rationale (proposer's reasoning):

> ['Tropical geometry provides a bridge to analyze algebraic properties of computational problems, which could reveal hidden complexities in circuit computation.', 'The Hodge index captures the complexity of the geometric structure of tropical varieties and might provide a natural invariant for characterizing the complexity class AC0.', 'Previous work on tropical geometry has shown its potential in understanding the complexity of arithmetic circuits. Extending this to AC0 parity circuits could offer new insights.']

Taxonomy category: `AC0_PARITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `798bd488055e1f93`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal tropical Hodge index of AC0 parity circuits is Ω(log^2(n)) if for at least 80% of 30 independent random seed generations, the mean tropical Hodge index across all circuits with n inputs is at or above log^2(n), and it is O(log(n)) otherwise.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.70	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ('tropical geometry' AND 'AC0 parity circuit') [title] - ('Hodge index' AND 'AC0 complexity') [abstract] - min tropical Hodge index AC0 circuit size [title]

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/1801.10489v3] Hodge level for weighted complete intersections - [http://arxiv.org/abs/2102.03481v3] Non-commutative Hodge conjecture - [http://arxiv.org/abs/2211.11405v4] On a Hodge locus - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, m):
                factor = -A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] += factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0]*p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
```

```
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += (-1)**j * A[0][j] * determinant(submatrix)
    return det

def tropical_divisor(gate, inputs):
    if gate == 'AND':
        return min(inputs)
    elif gate == 'OR':
        return max(inputs)
    elif gate == 'NOT':
        return 1 - inputs[0]
    else:
        raise ValueError("Invalid gate")

def tropical_variety(circuit):
    n = len(circuit)
    m = 2**n
    A = [[0]*m for _ in range(m)]
    b = [0]*m
    for i in range(m):
        inputs = [(i >> j) & 1 for j in range(n)]
        output = circuit(inputs)
        for j in range(m):
            if (j >> i) & 1:
                A[i][j] = tropical_divisor(circuit[0], [inputs[k] for k in range(1, n)])
                b[j] += output
    return gaussian_elimination(A), b

def tropical_hodge_index(A, b):
    det_A = determinant(A)
    if det_A == 0:
        return float('inf')
    return -math.log2(abs(det_A))

def ac0_circuit(n):
    gates = ['AND', 'OR', 'NOT']
    circuit = [random.choice(gates) for _ in range(1, n)]
    return circuit

def compute_tropical_hodge_index(circuit):
    A, b = tropical_variety(circuit)
    hodge_indices = []
    for i in range(len(b)):
        if b[i] != 0:
            hodge_indices.append(tropical_hodge_index(A, [b[j]/b[i] for j in range(len(b))]))
    return min(hodge_indices) if hodge_indices else float('inf')
```

```
    n_values = [5, 10, 15, 20, 30, 40]
    total_hodge_index = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            circuit = ac0_circuit(n)
            hodge_index = compute_tropical_hodge_index(circuit)
            total_hodge_index += hodge_index
            instances_tested += 1

    mean_hodge_index = total_hodge_index / instances_tested
    conjecture_holds = mean_hodge_index >= math.log2(n)**2 and mean_hodge_index <= math.log2(n)

    return {
        "metric_name": "tropical_hodge_index",
        "metric_value": mean_hodge_index,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_hodge_index = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_hodge_index} std=0 support_fraction={support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f30dd27.py", line 125, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f30dd27.py", line 104, in run_trial
```

```
    hodge_index = compute_tropical_hodge_index(circuit)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f30dd27.py", line 90, in compute_tropi
    A, b = tropical_variety(circuit)
           ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7f30dd27.py", line 71, in tropical_vari
    output = circuit(inputs)
             ^^^^^^^^^^^^^^^
TypeError: 'list' object is not callable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces the required data.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16003
2	propose	ollama_remote	glm4:latest	0	0	13202
3	preregistration	ollama_remote	glm4:latest	0	0	9521
4	novelty	ollama_remote	glm4:latest	0	0	8294
5	novelty	ollama_remote	glm4:latest	0	0	9205
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10653
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7829
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6700
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13806
10	judge	ollama_remote	glm4:latest	0	0	11551

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 106764 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f9751ccef221.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f9751ccef221.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f9751ccef221.tar.gz` (if generated)